

Website Availability Testing with curl

Objective

The objective of this lab is to learn how to use the curl command to verify website availability, inspect HTTP response headers, measure response times, and perform basic web service monitoring.

Tools Used

Commands used in this study:

- curl

Step 1 – Website Availability Test

The command was executed: curl -I https://www.google.com

Output:

```
curl -I https://www.google.com
```

```
HTTP/1.1 200 OK
```

```
Content-Type: text/html; charset=ISO-8859-1
```

```
Date: Sun, 09 Aug 2026 09:53:01 GMT
```

```
Server: gws
```

```
X-Frame-Options: SAMEORIGIN
```

```
Cache-Control: private
```

```
Set-Cookie: [redacted]
```

```
Alt-Svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000
```

Findings:

The curl -I command was used to inspect the HTTP response headers returned by Google. The server responded with HTTP/1.1 200 OK, indicating that the HTTP request was successfully processed. The Content-Type header showed that the response contained HTML content. The response also included server, caching, security, and cookie-related headers, providing additional information about how the web service handles the request.

Analysis:

The HTTP status code provides an initial indication of website availability. In this test, the 200 OK response confirmed that the web server successfully handled the request. Examining the response headers also provided information about the content type, server configuration, caching behavior, and

security-related settings. This type of HTTP header inspection can be useful for basic web service monitoring and troubleshooting.

Step 2 – HTTP Header Comparison

Command 1

```
curl -I https://github.com
```

Output (Representative Excerpt):

HTTP/1.1 200 OK

Date: Tue, 11 Aug 2026 11:21:37 GMT

Content-Type: text/html; charset=utf-8

Cache-Control: max-age=0, private, must-revalidate

Server: github.com

Command 2

```
curl -I https://www.microsoft.com
```

Output (Representative Excerpt):

HTTP/1.1 200 OK

Server: AkamaiGHost

Content-Type: text/html

Content-Length: 421

Cache-Control: max-age=0, no-cache, no-store

Date: Tue, 11 Aug 2026 11:23:12 GMT

Connection: close

Findings:

Both GitHub and Microsoft successfully responded to the HTTP HEAD requests with a 200 OK status code, indicating that the requests were successfully processed. Differences were observed in several response headers. GitHub reported github.com as the server, while Microsoft reported AkamaiGHost. The Content-Type header indicated HTML content for both websites, while GitHub additionally specified the UTF-8 character encoding. The Cache-Control headers also differed, showing that the two services use different cache-control configurations.

Analysis:

The comparison demonstrates that a successful HTTP response does not require identical server configurations. Both websites returned 200 OK, but their response headers differed in terms of server information, content type details, and cache-control policies. Inspecting these headers can help a network or operations engineer identify differences in web service configurations and investigate unexpected HTTP behavior during basic troubleshooting.

Step 3 – Redirect Test**Command 1:**

```
curl -I http://google.com
```

Output (Representative Excerpt):

HTTP/1.1 301 Moved Permanently

Location: <http://www.google.com/>

Command 2:

```
curl -I http://www.google.com
```

Output (Representative Excerpt):

HTTP/1.1 200 OK

Content-Type: text/html; charset=ISO-8859-1

Server: gws

Findings:

The initial request to <http://google.com> returned a 301 Moved Permanently response. The Location header indicated that the request was redirected to <http://www.google.com/>. A second request to the redirected URL returned 200 OK, indicating that the target URL was successfully accessible. Therefore, the redirect chain observed in this test was from google.com to www.google.com, rather than from HTTP to HTTPS.

Analysis:

The test demonstrated how HTTP redirects can be identified using the Location response header. The 301 Moved Permanently status indicated that the original URL redirected the client to another address. The successful 200 OK response from the redirected URL confirmed that the target resource was reachable. This type of redirect testing can be useful in network and web service troubleshooting when investigating URL accessibility and redirection behavior.

Step 4 – Measure Response Time

The command was executed: `curl -o NUL -s -w "DNS Lookup: %{time_namelookup}s\nConnect: %{time_connect}s\nSSL: %{time_appconnect}s\nTTFB: %{time_starttransfer}s\nTotal: %{time_total}s\n" https://www.google.com`

Output:

Test	DNS Lookup	Connect	SSL	TTFB	Total
1	0.024770 s	0.049852 s	0.145803 s	0.382382 s	0.458750 s
2	0.023867 s	0.051935 s	0.142587 s	0.378626 s	0.465349 s
3	0.009036 s	0.035622 s	0.111994 s	0.347536 s	0.444985 s
4	0.006349 s	0.035871 s	0.119702 s	0.389265 s	0.474315 s
5	0.005814 s	0.031486 s	0.103046 s	0.340431 s	0.417260 s
Average	0.013967 s	0.040953 s	0.124626 s	0.367648 s	0.452132 s

Findings:

The response time of `https://www.google.com` was measured five times using `curl`. The average DNS lookup time was approximately **0.014 seconds**, while the average TCP connection time was approximately **0.041 seconds**. The average SSL/TLS connection time was approximately **0.125 seconds**.

The average Time To First Byte (TTFB) was approximately **0.368 seconds**, and the average total request time was approximately **0.452 seconds**. The total response time across the five tests ranged from **0.417 seconds to 0.474 seconds**.

Analysis:

The measurements show that the response time remained relatively consistent across the five tests, with only minor variations. DNS lookup and TCP connection establishment accounted for relatively small portions of the overall request time, while SSL/TLS negotiation and server response contributed more significantly.

The average TTFB was approximately 368 ms, while the average total request time was approximately 452 ms. Repeating the measurements demonstrated that response times can vary between requests, highlighting the importance of monitoring multiple measurements during network troubleshooting.

Step 5 – Error Test

Command 1:

```
curl -I https://httpbingo.org/status/404
```

Output (Representative Excerpt):

HTTP/1.1 404 Not Found

content-type: text/plain; charset=utf-8

date: Tue, 11 Aug 2026 13:59:53 GMT

server: Fly/d42146e16 (2026-08-06)

Command 2:

```
curl -I https://httpbingo.org/status/503
```

Output (Representative Excerpt):

HTTP/1.1 503 Service Unavailable

content-type: text/plain; charset=utf-8

date: Tue, 11 Aug 2026 14:00:47 GMT

server: Fly/d42146e16 (2026-08-06)

Test	HTTP Status	Meaning
/status/404	404 Not Found	Requested resource could not be found
/status/503	503 Service Unavailable	Server is temporarily unable to handle the request

Findings:

Two HTTP error scenarios were tested using `curl`: a 404 Not Found response and a 503 Service Unavailable response. The request to `/status/404` returned HTTP 404, indicating that the requested resource could not be found. The request to `/status/503` returned HTTP 503, indicating that the service was temporarily unavailable.

Both responses included HTTP headers such as `Content-Type`, `Date`, and `Server`. The tests demonstrated that different HTTP status codes provide information about the type of problem encountered when accessing a web service.

Analysis:

The results show that HTTP error codes can be used to distinguish between different types of problems. A 404 response generally indicates that the requested resource could not be found, while a 503 response indicates that the server or service is temporarily unable to handle the request.

From a NOC perspective, these responses should be interpreted differently during troubleshooting. A 404 error may require checking the requested URL or resource, whereas a 503 error may require investigating the availability and operational status of the service. Therefore, examining HTTP status codes can help identify the direction of further troubleshooting.

Key Concepts Learned

During this exercise, the following concepts were reinforced:

- HTTP Status Codes
- HTTP Response Headers
- Website Availability Testing
- HTTP 200 OK Response
- HTTP 301 Redirects
- HTTP 404 Not Found
- HTTP 503 Service Unavailable
- HTTP Header Comparison
- DNS Lookup Time
- TCP Connection Time
- SSL/TLS Connection Time
- Time To First Byte (TTFB)
- Total Response Time
- Basic Web Service Monitoring
- Basic Network Troubleshooting
- Client-side vs. Server-side Error Interpretation

Conclusion

This lab provided practical experience with using `curl` to test website availability, inspect HTTP response headers, analyze redirects, measure response times, and identify different HTTP error conditions. The tests demonstrated how HTTP status codes and response headers can provide useful information about the behavior and availability of web services.

The response-time measurements also showed how DNS lookup, TCP connection establishment, SSL/TLS negotiation, and server response contribute to the overall request time. Repeating the measurements helped demonstrate that response times can vary between requests and should be monitored over multiple tests.

Overall, the lab strengthened practical skills that can be applied in NOC environments, particularly for basic website monitoring, service availability checks, response-time analysis, and initial troubleshooting of web-related network issues.